AI Critique

During this assignment, the AI demonstrated impressive speed in generating Playwright boilerplate, parsing test data, and analyzing error logs. However, it exhibited significant flaws regarding contextual awareness and testing principles. The most critical error was the AI’s tendency to select DOM locators based on the actual web implementation rather than the Software Requirements Specification. For instance, if the specification required an "Email" field but the developer mistakenly implemented a "Username" field, the AI would adjust its script to target "Username" simply to make the test pass. Furthermore, it struggled with environmental nuances, such as misconfiguring IPv6 network settings and failing to implement database teardowns between test runs, which led to cross-test state pollution and intermittent script failures before data-driven loops could even execute

Why did the AI fail to catch these issues? Primarily, LLM are trained to solve immediate problems and tend to favor "green" execution outcomes. The AI lacks an inherent "Testing Mindset"; it does not naturally understand that a test's primary goal is to expose application defects by strictly enforcing the SRS. When faced with a mismatch between the specification and the current DOM, its default behavior was to accommodate the DOM. Additionally, without explicit prompting, it cannot anticipate backend state persistence issues that occur across parallel automated test executions

This experience reinforced a vital principle: **AI is an efficient code generator, but the human must remain the test architect.** Collaborating with AI requires strict human-in-the-loop oversight. I learned that I cannot blindly trust AI-generated locators or environment configurations. As a QA engineer, I must meticulously review the scripts to ensure they enforce the original requirements, not just the flawed UI, and I must manually dictate best practices for test isolation and state management